

SPFA with Tarjan’s Subtree Disassembly Trick

Henghui Zhu 12412412

Abstract

This report describes an improved implementation of the Bellman–Ford–Moore algorithm, commonly known as SPFA, using Tarjan’s subtree disassembly trick. The algorithm maintains a pointer graph that records the current explanation of shortest-path labels. When a selected pointer changes, the algorithm checks whether a negative cycle is created and, if not, disassembles the obsolete subtree rooted at the updated vertex. This gives immediate negative-cycle detection and often a large practical speedup, while preserving the same worst-case time complexity $O(mn)$ and using only $O(n)$ extra working space.

1 Setting and Notation

Let $G = (V, E)$ be a directed graph with edge cost $c(u, v)$, where $|V| = n$ and $|E| = m$. We consider shortest paths to a fixed terminal node t . The algorithm should either compute shortest-path labels to t or report that a negative cycle exists.

The push-based Bellman–Ford–Moore algorithm keeps a current distance label

$$M[v] \approx \text{current best known distance from } v \text{ to } t.$$

Initially, $M[t] = 0$ and $M[v] = +\infty$ for all $v \neq t$. When the distance of a node w decreases, the algorithm scans all incoming edges (v, w) . If

$$M[v] > c(v, w) + M[w],$$

then $M[v]$ can be decreased, and v becomes active.

To support negative-cycle detection, we also maintain a pointer

$$\text{first}[v] = \text{the first node after } v \text{ on the current path realizing } M[v].$$

If $\text{first}[v] = w$, the selected pointer edge is (v, w) . These selected edges form a pointer graph P . When P is acyclic, it is a forest of directed paths toward t . If adding a new pointer edge creates a directed cycle in P , then that cycle must be a negative-cost cycle, because the pointer was created by a strict distance improvement.

Tarjan’s trick improves SPFA by detecting such cycles immediately and by deleting stale subtrees whose distance labels were derived from an old value.

2 Pseudocode

The pseudocode below uses incoming adjacency lists. For each vertex w , $\text{In}[w]$ stores all edges (v, w) entering w . Besides $\text{first}[v]$, the algorithm maintains reverse child lists in the pointer graph:

$$\text{children}[x] = \{y : \text{first}[y] = x\}.$$

These lists allow us to find all vertices in the subtree directed toward a vertex v .

Algorithm 1 SPFA with Tarjan's Subtree Disassembly Trick

Require: Directed graph $G = (V, E)$, edge cost $c(u, v)$, terminal node t

Ensure: Shortest-path labels $M[v]$ if no negative cycle is found; otherwise report a negative cycle

```

1: for all  $v \in V$  do
2:    $M[v] \leftarrow +\infty$ 
3:    $\text{first}[v] \leftarrow \text{NIL}$ 
4:    $\text{children}[v] \leftarrow \emptyset$ 
5:    $\text{dormant}[v] \leftarrow \text{FALSE}$ 
6:    $\text{inQueue}[v] \leftarrow \text{FALSE}$ 
7: end for
8:  $M[t] \leftarrow 0$ 
9:  $Q \leftarrow$  empty queue
10: enqueue  $t$  into  $Q$ ;  $\text{inQueue}[t] \leftarrow \text{TRUE}$ 
11: while  $Q$  is not empty do
12:    $w \leftarrow$  dequeue  $Q$ ;  $\text{inQueue}[w] \leftarrow \text{FALSE}$ 
13:   if  $\text{dormant}[w] = \text{TRUE}$  then
14:     continue
15:   end if
16:   for all edges  $(v, w) \in \text{In}[w]$  do
17:     if  $M[w] \neq +\infty$  and  $M[v] > c(v, w) + M[w]$  then
18:        $\text{newValue} \leftarrow c(v, w) + M[w]$ 
19:        $(\text{status}, C) \leftarrow \text{DISASSEMBLE-AND-CHECK}(v, w)$ 
20:       if  $\text{status} = \text{CYCLE}$  then
21:         return "negative cycle found",  $C$ 
22:       end if
23:        $M[v] \leftarrow \text{newValue}$ 
24:        $\text{first}[v] \leftarrow w$ 
25:       add  $v$  to  $\text{children}[w]$ 
26:        $\text{dormant}[v] \leftarrow \text{FALSE}$ 
27:       if  $\text{inQueue}[v] = \text{FALSE}$  then
28:         enqueue  $v$  into  $Q$ ;  $\text{inQueue}[v] \leftarrow \text{TRUE}$ 
29:       end if
30:     end if
31:   end for
32: end while
33: return  $M$ 

```

Algorithm 2 Subroutine DISASSEMBLE-AND-CHECK(v, w)

Require: The algorithm is about to replace $\text{first}[v]$ by w

Ensure: Either report the directed cycle created by (v, w) , or disassemble the old subtree rooted at v

```
1:  $S \leftarrow$  all vertices in the subtree rooted at  $v$  in the current pointer graph  $P$   $\triangleright$  Use DFS/BFS
   over the child lists
2: if  $w \in S$  then
3:    $C \leftarrow$  edge  $(v, w)$  plus the old pointer path from  $w$  back to  $v$ 
4:   return (CYCLE,  $C$ )
5: end if
6: for all  $x \in S$  do
7:   if  $\text{first}[x] \neq \text{NIL}$  then
8:     remove  $x$  from  $\text{children}[\text{first}[x]]$ 
9:   end if
10:   $\text{first}[x] \leftarrow \text{NIL}$ 
11:   $\text{children}[x] \leftarrow \emptyset$ 
12:   $\text{dormant}[x] \leftarrow \text{TRUE}$ 
13: end for
14: return (NO-CYCLE,  $\emptyset$ )
```

After DISASSEMBLE-AND-CHECK(v, w) returns NO-CYCLE, the main algorithm installs the new pointer $\text{first}[v] = w$, wakes up v , and continues the SPFA process.

3 Core Idea and Critical Procedures

3.1 The pointer graph records the current explanation of distances

Whenever $M[v]$ is improved through edge (v, w) , we set

$$\text{first}[v] = w.$$

This means the current value of $M[v]$ is explained by first taking edge (v, w) and then following the current pointer path from w to t .

If the pointer graph ever contains a directed cycle, then following the strict improvement inequalities around the cycle implies that the total cost of the cycle is negative. Therefore, detecting a cycle in the pointer graph is enough to detect a negative cycle in the original graph.

3.2 Naive instant cycle detection is too expensive

Suppose $\text{first}[v]$ is updated from its old value to w . A natural way to test whether this creates a cycle is to follow the old pointer path from w toward t . If this path reaches v , then adding (v, w) creates a cycle.

However, this path can have length $\Theta(n)$. If the algorithm performs such a path traversal after many relaxations, it may spend too much extra time.

3.3 Tarjan's trick: check the subtree of v , not the path from w

Before adding the new pointer edge (v, w) , the pointer graph is acyclic. Adding (v, w) creates a cycle exactly when w is already in the subtree rooted at v . Here, the subtree rooted at v means

all nodes whose current pointer path eventually reaches v .

Therefore, the algorithm performs the following steps:

1. Use reverse child lists to find the subtree S rooted at v .
2. If $w \in S$, report a negative cycle immediately.
3. Otherwise, delete all old pointer edges inside S and mark nodes in S as dormant.
4. Install the new pointer $\text{first}[v] = w$, wake v , and continue SPFA.

3.4 Why marking nodes dormant is safe

Every node x in the subtree rooted at v had a current distance label that depended on the old value of $M[v]$. Once $M[v]$ decreases, these old explanations become stale. These nodes will need to be updated again only if a better path is actually propagated to them later.

Instead of allowing all of them to remain active and push obsolete information, Tarjan's trick marks them as dormant. A dormant node is not used to push relaxations. It becomes active again only when its own distance label is improved later.

This is the key insight: the work spent traversing and disassembling the subtree is not wasted. It also prevents future useless queue processing.

4 Complexity Analysis

4.1 Space complexity

The algorithm stores the following auxiliary data structures:

- the distance array $M[]$;
- the pointer array $\text{first}[]$;
- queue flags $\text{inQueue}[]$;
- dormant flags $\text{dormant}[]$;
- the queue;
- child-list information for the pointer graph.

Obviously, $M[]$, $\text{first}[]$, $\text{inQueue}[]$, $\text{dormant}[]$, the queue takes $O(n)$.

For $\text{children}[]$, $\text{children}[v]$ stores all the u that $\text{first}[u]=v$, while every u has only 1 $\text{first}[u]$. So the space complexity for $\text{children}[]$ is $O(n)$. What's more, every $\text{first}[]$ update depends on an edge in the graph, so the space complexity for $\text{children}[]$ is also $O(m)$. Therefore, the complexity for $\text{children}[]$ is $O(\min\{m, n\})$.

Including the graph storage, the total space is $O(n + m)$.

4.2 Time complexity

The worst-case running time remains

$$O(mn).$$

Ignoring the time spent disassembling subtrees, the algorithm is still a Bellman–Ford–Moore/SPFA-style algorithm. In the layered interpretation, after i effective rounds, the algorithm has correctly

handled all shortest paths that can be represented by simple paths of at most i edges. If no negative cycle is found, a simple path has at most $n - 1$ edges, so there are at most n effective rounds. Each round may scan $O(m)$ edges, giving $O(mn)$.

Now consider the extra time spent by Tarjan's disassembly procedure. A node can be marked dormant only after it has previously received a selected pointer. Receiving such a pointer happens together with a distance update. Therefore, the total amount of subtree-disassembly work can be charged to previous distance updates. Since the number of such updates is bounded by the same $O(mn)$ worst-case analysis, disassembly does not increase the asymptotic worst-case time.

Thus,

$$\text{worst-case time} = O(mn), \quad \text{extra space} = O(n).$$

The main benefit is not an asymptotic worst-case improvement, but a large practical speedup.

5 Why Tarjan's Trick Can Be Much Faster Without Negative Cycles

Tarjan's trick helps even when the graph has no negative cycle because SPFA may still create many temporary distance labels that are later improved. These temporary labels can cause large amounts of useless queue processing.

5.1 Example 1: a long chain of stale descendants

Consider a graph in which a long chain of vertices depends on the current label of v :

$$x_k \rightarrow x_{k-1} \rightarrow \cdots \rightarrow x_2 \rightarrow x_1 \rightarrow v \rightarrow t.$$

Suppose SPFA first discovers a path from v to t with cost 100. Then this value may be propagated backward, giving labels such as

$$M[x_1] = 100, \quad M[x_2] = 100, \quad \dots, \quad M[x_k] = 100,$$

up to additive edge costs. The pointer graph then contains a large subtree rooted at v .

Later, suppose the algorithm discovers a better path

$$v \rightarrow w \rightarrow t$$

with total cost 1. Then $M[v]$ decreases from 100 to 1.

At this moment, all nodes $x_1, \dots, x_k$ have stale labels, because their values were derived from the old value $M[v] = 100$. In ordinary SPFA, many of these nodes may still be in the queue. They may be popped, scan their incoming edges, and push more obsolete updates before they are corrected later.

With Tarjan's trick, when $\text{first}[v]$ changes, the algorithm finds the subtree rooted at v , marks $x_1, \dots, x_k$ dormant, and removes their old pointers. These nodes will not perform useless relaxations. They will wake up only when the new smaller value actually reaches them.

This graph has no negative cycle; all edge costs in the example can be nonnegative. The speedup comes purely from avoiding stale work.

5.2 Example 2: a bushy subtree with high in-degree nodes

Suppose v has a large pointer subtree containing K nodes, and each of these nodes has many incoming edges. If $M[v]$ decreases, the labels of all K descendants become obsolete.

Without Tarjan's trick, SPFA may process many of those K stale nodes. If each stale node scans many incoming edges, the wasted work can be very large.

With Tarjan's trick, the algorithm pays only $O(K)$ time to disassemble the subtree and mark these nodes dormant. This can avoid scanning a much larger number of incident edges. Therefore, the practical speedup can be considerable.

5.3 Example 3: avoiding repeated waves of obsolete propagation

In many graphs, distance labels improve in several waves. A node may first receive a mediocre path, then a better path, then an even better path. In ordinary SPFA, each wave may propagate far into the graph before being replaced by the next wave.

Tarjan's trick stops old waves early. Whenever a root node gets a better successor, its old dependent subtree is cut off. This prevents the algorithm from continuing to propagate information that is already known to be outdated.

Therefore, even when no negative cycle exists, Tarjan's trick can make SPFA much faster in practice by:

- detecting negative cycles immediately if they appear in the pointer graph;
- preventing dormant nodes from pushing stale relaxations;
- avoiding repeated scans of edges caused by obsolete distance labels;
- replacing large amounts of future useless work with one subtree traversal.

6 Conclusion

Tarjan's subtree disassembly trick improves SPFA by maintaining the pointer graph of current shortest-path explanations. When a pointer $\text{first}[v]$ is changed, the algorithm checks the old subtree rooted at v . If the new successor lies inside this subtree, a negative cycle is found immediately. Otherwise, the old subtree is disassembled and marked dormant, because its labels depend on the old value of $M[v]$.

The algorithm still has worst-case time complexity $O(mn)$ and only $O(n)$ extra space, but in practice it can be much faster than ordinary SPFA. Its main advantage is that it avoids processing stale queue entries and obsolete pointer subtrees.